

ChatGPT



```
class Solution {
    public int[] quickSort(int[] nums) {
        int n = nums.length;
        int low=0;
        int high = n-1;
        quickS(nums,low,high);
        return nums;
    }
    private void quickS(int[] nums,int low,int high){
        if(low<high){
            int partitionIndex=pivotFinder(nums,low,high);
            quickS(nums,low,partitionIndex-1);
            quickS(nums,partitionIndex+1,high);
        }
    }

    private int pivotFinder(int nums[],int low, int high){
        int pivot=nums[low];
        int i=low;
        int j=high;

        while(i<j){
            while(i<=high && nums[i]<=pivot ){
                i++;
            }
            while(j>=low && nums[j]>pivot ){
                j--;
            }
            if(i<j){
                int temp = nums[i];
                nums[i] = nums[j];
                nums[j] = temp;
            }
        }
        int temp = nums[low];
        nums[low] = nums[j];
        nums[j] = temp;
        return j;
    }
}
```

Teach me Quick Sort
 Show moreShow less

Absolutely. Your implementation is a **Hoare-style partition Quick Sort** where the pivot is the first element. Let's understand the algorithm from the idea → partition → your code → dry run → complexity.

1. What is Quick Sort?

Quick Sort is a **divide-and-conquer** sorting algorithm.

The main idea is:

1. Pick an element as a **pivot**.
2. Rearrange the array so:
 - elements smaller/equal to pivot go to the left
 - elements greater than pivot go to the right
3. The pivot reaches its correct position.
4. Recursively do the same thing on the left and right portions.

For example:

```
[5, 2, 8, 1, 7, 3]
pivot = 5
[2, 1, 3] 5 [8, 7]
  left  pivot  right
```

Then sort the two sides:

```
[2, 1, 3]    [8, 7]
  ↓          ↓
[1, 2, 3]    [7, 8]
```

Final:

```
[1, 2, 3, 5, 7, 8]
```

2. Your Quick Sort structure

Your code has three important methods:

```
Java
quickSort()
  ↓
quickS()
  ↓
pivotFinder()
```

Think of their responsibilities like this:

quickSort()

Starts the algorithm.

```
Java
quickS(nums, 0, n - 1);
```

quickS()

Recursively sorts the left and right portions.

```
Java
int partitionIndex = pivotFinder(nums, low, high);

quickS(nums, low, partitionIndex - 1);
quickS(nums, partitionIndex + 1, high);
```

pivotFinder()

Does the actual partitioning.

This is the part you should understand most carefully.

3. Why do we need low and high?

Suppose:

```
[5, 2, 8, 1, 7, 3]
```

Initially:

```
low = 0
high = 5
```

So we're sorting:

```
[5, 2, 8, 1, 7, 3]
  ↑       ↑
  low    high
```

After the pivot 5 is placed correctly, suppose we get:

```
[3, 2, 1, 5, 7, 8]
      ↑
    pivot
```

Now we don't need to touch 5.

So we recursively sort:

```
[3, 2, 1]    [7, 8]
```

That's why:

```
Java
```

```
quickS(nums, low, partitionIndex - 1);
quickS(nums, partitionIndex + 1, high);
```

4. Your pivot

You choose:

```
Java
int pivot = nums[low];
```

So your pivot is always the **first element** of the current subarray.

For:

```
[5, 2, 8, 1, 7, 3]
```

we have:

```
pivot = 5
```

5. The two pointers: i and j

This is the key part.

You start with:

```
Java
int i = low;
int j = high;
```

Visually:

```
[5, 2, 8, 1, 7, 3]
  ↑         ↑
  i         j
```

i moves **from left** → **right**.

j moves **from right** → **left**.

Their job:

- i looks for an element that is **too large** and belongs on the right.
- j looks for an element that is **small enough** and belongs on the left.

6. Understanding this loop

You have:

```
Java
while(i <= high && nums[i] <= pivot){
    i++;
```

```
}
```

Translation:

Keep moving `i` right while the current element belongs on the left side.

For:

```
[5, 2, 8, 1, 7, 3]
  ↑  ↑
pivot i
```

`2 <= 5`, so move `i`.

```
[5, 2, 8, 1, 7, 3]
      ↑
      i
```

Now:

```
8 > 5
```

Stop.

So `i` has found:

```
8
```

which is too large for the left side.

7. Now `j`

Your code:

```
Java
while(j >= low && nums[j] > pivot){
    j--;
}
```

Translation:

Keep moving `j` left while the element belongs on the right side.

Starting:

```
[5, 2, 8, 1, 7, 3]
          ↑
          j
```

`3 > 5?`

No.

So `j` stops immediately.

We now have:

```
i → 8
j → 3
```

These are misplaced relative to the pivot:

```
8 should go right
3 should go left
```

So swap them.

```
[5, 2, 3, 1, 7, 8]
```

8. Continue

Now:

```
i → 8
j → 3
```

The loops continue.

i moves left-to-right until it finds something greater than 5.

It eventually finds:

```
7
```

j moves right-to-left until it finds something ≤ 5 .

It finds:

```
1
```

Swap:

```
[5, 2, 3, 1, 7, 8]
      ↓      ↓
      1      7
```

Actually, after the pointer movements, the swap produces:

```
[5, 2, 3, 1, 7, 8]
```

At this point *i* has crossed *j*, so partitioning stops.

The important invariant is:

```
elements on left side ≤ pivot ≤ elements on right side
```

9. Why do we finally swap `nums[low]` and `nums[j]`?

This is the most important line:

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

Java

```
int temp = nums[low];
nums[low] = nums[j];
nums[j] = temp;
```

Remember:

Java

```
pivot = nums[low];
```

The pivot is still sitting at `low`.

After `i` and `j` cross, `j` tells us where the pivot belongs.

So we put the pivot there.

Conceptually:

```
[ pivot | elements ≤ pivot | elements > pivot ]
  ↑
  low

                ↑
                j
```

Swap pivot with `nums[j]`.

Then:

```
[ elements ≤ pivot | pivot | elements > pivot ]
                      ↑
                correct position
```

And:

Java

```
return j;
```

returns the pivot's final position.

10. The recursive part

Suppose partition gives:

```
[3, 2, 1, 5, 7, 8]
      ↑
    index 3
```

`partitionIndex = 3.`

Then:

Java

```
quickS(nums, low, partitionIndex - 1);
```

means:

```
sort indices 0 → 2
[3, 2, 1]
```

And:

```
Java
quickS(nums, partitionIndex + 1, high);
```

means:

```
sort indices 4 → 5
[7, 8]
```

Notice:

We never include index 3 again.

Why?

Because the pivot is already in its correct position.

11. The base condition

You have:

```
Java
if(low < high)
```

This is extremely important.

When:

```
low == high
```

there is only one element.

Example:

```
[7]
```

A single element is already sorted.

When:

```
low > high
```

there's nothing to sort.

Therefore recursion stops.

12. Let's dry-run a small example

Take:

```
[4, 2, 7, 1, 3]
```

First call

```
low = 0
high = 4
pivot = 4
```

We want:

```
smaller than 4 | 4 | greater than 4
```

Partition rearranges it to something like:

```
[3, 2, 1, 4, 7]
```

Pivot 4 is now at index 3.

So:

```
Java
quickS(nums, 0, 2);
quickS(nums, 4, 4);
```

Right side has one element → done.

Sort left side

```
[3, 2, 1]
```

Pivot:

```
3
```

Partition:

```
[1, 2, 3]
```

Now 3 is correct.

Sort:

```
[1, 2]
```

Pivot:

```
1
```

Already effectively partitioned:

```
[1, 2]
```

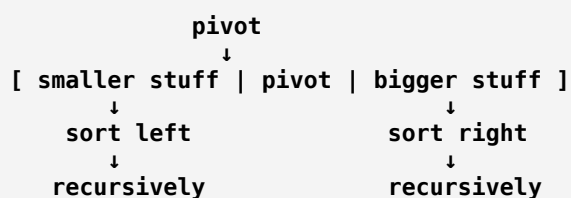
Final:

```
[1, 2, 3, 4, 7]
```

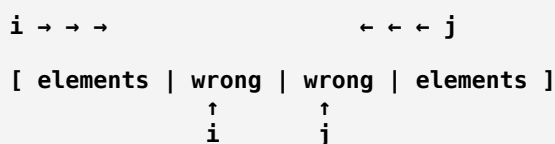
13. The mental model I want you to remember

Don't memorize the code.

Remember this picture:



And during partition:



i finds a value that is too big.

j finds a value that is small enough.

Swap them.

Repeat.

When they cross, put the pivot at *j*.

14. One subtle point in YOUR implementation

You wrote:

```
Java
int i = low;
```

Since:

```
Java
pivot = nums[low];
```

the first check is:

```
Java
```

```
nums[i] <= pivot
```

which is:

```
Java
pivot <= pivot
```

so *i* immediately moves forward.

That's perfectly fine.

Similarly, your *j* starts at **high**.

This implementation is a valid partition approach.

You could also start:

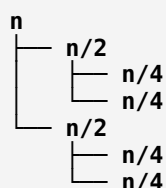
```
Java
int i = low + 1;
```

because we already know the pivot itself doesn't need to be checked.

15. Time complexity

Average case

Quick Sort usually divides the array reasonably well:



So:

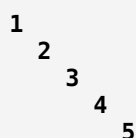
Average: $O(n \log n)$

Worst case

If you repeatedly choose a terrible pivot, for example:

```
[1, 2, 3, 4, 5, 6, 7]
```

and always choose the first element:



you get:

Worst case: $O(n^2)$

That's a weakness of choosing the first element as pivot.

Space

Quick Sort is recursive, so the recursion stack uses:

- Average: $O(\log n)$
- Worst: $O(n)$

Your algorithm itself is **in-place**, because you're rearranging the original array rather than creating another array.

The 3 things you should master

If you're learning this for coding interviews, focus on these three things:

1. **What does the pivot mean?**

→ After partitioning, the pivot is in its final sorted position.

2. **What are `i` and `j` doing?**

→ `i` finds an element that belongs right; `j` finds one that belongs left.

3. **Why `partitionIndex - 1` and `partitionIndex + 1`?**

→ Because the pivot at `partitionIndex` is already sorted.

Once those three click, Quick Sort becomes much less mysterious.

If you want, I can next teach you `pivotFinder()` **line-by-line with a pointer-by-pointer dry run on [5, 2, 8, 1, 7, 3]**, which is probably the best way to make this implementation stick.

ChatGPT can make mistakes. Check important info.